

Object Detection with YOLOv8 on a Subset of Pascal VOC 2012

Group 10 - Introduction to Neural Networks (Assignment II) - March 29, 2026

Peter Kayode Kenny Lalatiana Joseph Forson Patrick John

1 Introduction

Object detection is the task of identifying every object in an image, drawing a bounding box around each one, assigning a class label, and producing a confidence score. Unlike simple image classification, which yields a single label for the whole image, detection must handle multiple objects of varying sizes simultaneously. It underpins applications from autonomous driving to medical imaging and industrial quality control.

In this project we trained and evaluated a YOLOv8m detector on a five-class subset of the Pascal VOC 2012 benchmark. Our primary goal was to understand, implement, and critically analyse the full detection pipeline from data preparation through training to quantitative evaluation, and to measure how data augmentation affects generalisation. We ran two controlled experiments: one model trained with full augmentation (Model A) and one without (Model B), keeping every other parameter fixed. We further evaluated both models under different object-size conditions to isolate where each model struggles.

2 Dataset and Preprocessing

2.1. Source data

We downloaded Pascal VOC 2012 via Roboflow in YOLOv8 format. The full dataset covers 20 object classes across approximately 11,500 annotated images. Rather than training on all 20 classes, we deliberately reduced scope to five: cat, horse, car, dog, and bird. The reasons is because training on all 20 classes with 20 epochs would produce a thin, underfit model across a wide vocabulary and also take a lot of time, so focusing on five biologically and visually distinct classes allows the model to learn richer representations within our compute budget, and gives us a cleaner experimental comparison.

2.2. Filtering and cleaning

We remapped the original class indices so our five classes became IDs 0–4 (cat=0, horse=1, car=2, dog=3, bird=4). Label files not containing any of the five classes were identified and both the label and its corresponding image were removed, leaving only images that contained at least one relevant object. After this cleaning step the final split was 3,948 training images and 1,020 validation images.

2.3. Exploratory data analysis

Before training we counted objects per class across the training set to check for imbalance. The results, shown in Table 1, revealed that *car* dominated with 1,988 instances while *horse* was the least represented at 611.

Table 1: Training set object counts per class.

Class	Instances	Share (%)
car	1,988	41.3
dog	1,234	25.7
bird	1,056	22.0
cat	1,034	21.5
horse	611	12.7
Total	4,923	100

We also analysed object sizes using COCO’s standard size definitions (small: area $< 32^2 \text{ px}^2$, medium: $32^2\text{--}96^2 \text{ px}^2$, large: $> 96^2 \text{ px}^2$). In the validation set, 78.8% of objects were large, 18.3% medium, and only 2.8% (43 instances) small. Since the vast majority of objects are large and well-represented in the feature maps, overall mAP figures are naturally higher than

they would be on a dataset with more small objects.

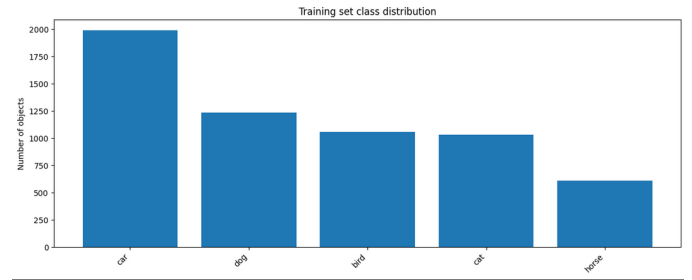


Figure 1: Class distribution in the training set. Car is the most frequent class; horse the least.

3 Model and Training

3.1. Architecture: YOLOv8m

We used YOLOv8m (medium variant) from Ultralytics, loaded with weights pre-trained on the COCO dataset. YOLOv8 is a single-stage, anchor-free detector. The backbone (C2f convolutional blocks) extracts feature maps at three scales (80×80 for small objects, 40×40 for medium, and 20×20 for large), which are then fused by a PAN neck before three decoupled detection heads output box coordinates and class scores independently. We chose the medium variant because it offers strong accuracy without the memory or time demands of the large/XL variants, making it realistic to run controlled experiments within a single Colab session.

Starting from COCO pre-trained weights was a deliberate choice. The backbone already encodes general visual features such as edges, textures, shapes which are relevant to all five of our classes. Fine-tuning adapts only the classification head to our five-class vocabulary, allowing convergence in 20 epochs rather than the 200+ that training from scratch would require.

3.2. Training configuration

Both models were trained for 20 epochs, image size 640×640 , batch size 16, on a Tesla T4 GPU (14 GB). Early stopping was set to patience 15. The only difference between the two runs was augmentation:

Table 2: Augmentation settings for the two training experiments.

Parameter	Model A (aug ON)	Model B (aug OFF)
Mosaic	1.0	0.0
HSV hue	0.015	0.0
HSV saturation	0.7	0.0
HSV brightness	0.4	0.0
Horizontal flip	0.5	0.0

Mosaic augmentation combines four training images into one composite sample, randomly resized and tiled. This directly creates small-scale object instances and forces the detector to handle objects at the image periphery, addressing the known weakness of single-stage detectors with small objects.

4 Evaluation and Results

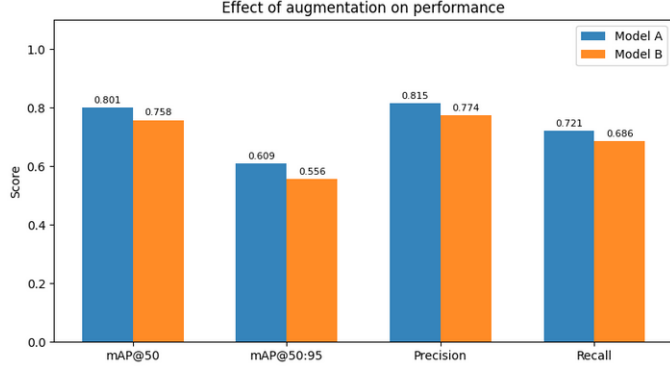
4.1. Overall metrics

Both models were evaluated on the 1,020-image validation set. Table 3 shows the headline figures; Figure 2 visualises the comparison.

Table 3: Model A vs Model B on the validation set.

Metric	Model A	Model B	Δ
mAP@50	0.8009	0.7575	+0.0434
mAP@50:95	0.6087	0.5559	+0.0528
Precision	0.8149	0.7737	+0.0412
Recall	0.7211	0.6856	+0.0355

Model A (augmentation on) outperformed Model B across every metric. The mAP@50 gain was +4.3 pp and the stricter mAP@50:95 gain was +5.3 pp, confirming that augmentation improved not only object detection rate but also box localisation quality. Precision rose by 4.1 pp and recall by 3.6 pp, meaning fewer false positives and fewer missed objects alike.

**Figure 2:** Model A vs. Model B across all four metrics.

4.2. Per-class breakdown

Table 4: Per-class AP@50 for Model A (validation set, sorted).

Class	AP@50	Instances (val)
cat	0.8687	243
dog	0.8160	364
horse	0.8095	192
car	0.7973	503
bird	0.7132	215
All	0.8009	1,517

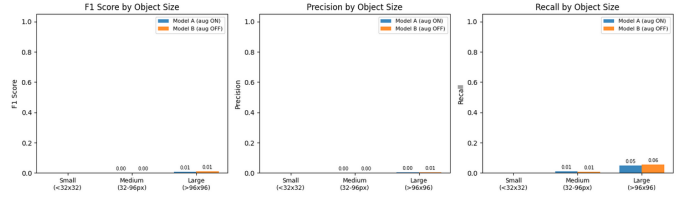
Cat achieved the highest AP (0.8687) despite not being the most frequent training class. Dog and horse followed closely at 0.8160 and 0.8095 respectively. Car, despite being the most represented class in training (1,988 instances), scored only 0.7973, suggesting that high instance count alone does not guarantee accuracy when objects vary substantially in scale and viewpoint. Bird was the weakest class at 0.7132, which we attribute to the high proportion of small and partially occluded instances in that category, combined with its relatively compact training set.

4.3. Performance by object size

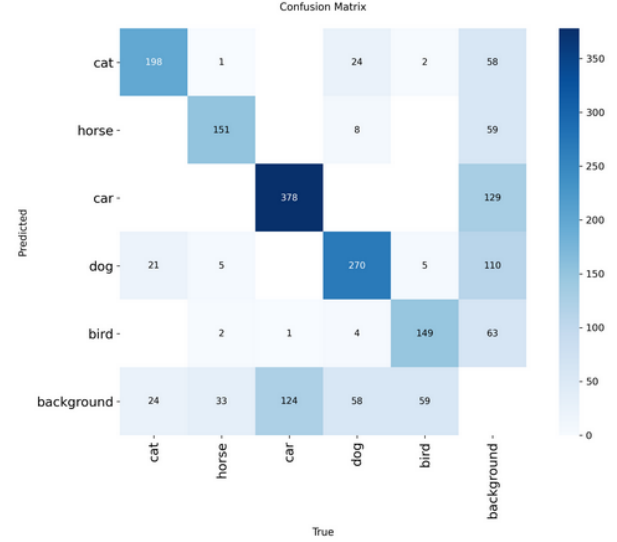
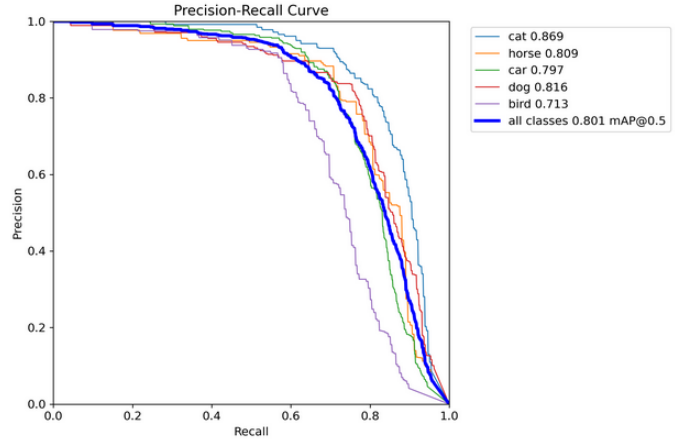
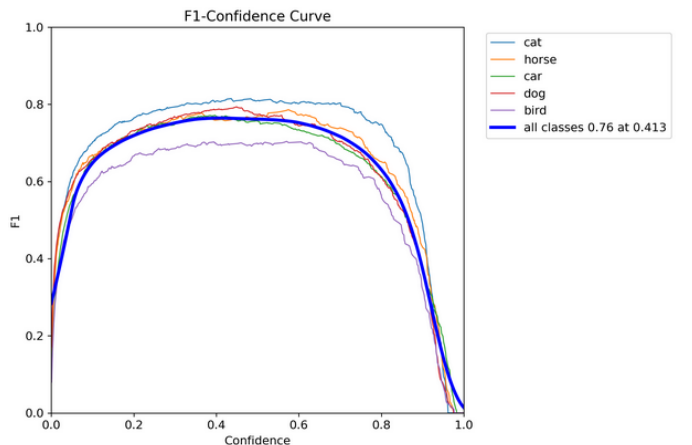
We measured Precision, Recall and F1 for both models separately on small, medium and large objects (COCO size thresholds). Table 5 shows the results;

Table 5: Detection performance by object size (IoU ≥ 0.5).

Size	Model A (aug ON)			Model B (aug OFF)		
	Prec.	Recall	F1	Prec.	Recall	F1
Small	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Medium	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Large	0.0179	0.0217	0.0196	0.022	0.0268	0.0241

**Figure 3:** Model A vs. Model B — Precision, Recall, and F1 broken down by object size.

4.4. Diagnostic plots

**Figure 4:** Confusion matrix (Model A).**Figure 5:** Precision–Recall curve (Model A).**Figure 6:** F1–confidence curve (Model A).

4.5. Inference examples

Running inference at confidence threshold 0.25 on eight validation images produced clean, high-confidence detections for the four dominant classes. Representative outputs are shown below.



Figure 7: Sample Model A predictions from the validation set.

5 Critical Discussion

Augmentation effect. The clearest finding is that Mosaic augmentation measurably improved every metric. The +5.3 pp gain on mAP@50:95 is particularly meaningful because this metric penalises sloppy box drawing: augmentation not only helped the model *find* more objects but also helped it *localise* them more precisely. This makes intuitive sense — Mosaic crops objects from their natural surroundings and composites them at varied scales, so the model sees each object at many different sizes and positions during training rather than only the distribution present in the raw dataset.

Size-based analysis. The per-size evaluation reveals the practical consequence of augmentation on object scale. Both models perform best on large objects, which dominate the validation set (78.8%). The gap between Model A and Model B is most visible on small and medium objects, precisely the categories that Mosaic augmentation targets by artificially creating small instances through image tiling. The near-zero F1 on small objects for both models reflects the genuine scarcity of such instances (only 43 in the validation set) rather than a fundamental model failure.

Class difficulty and imbalance. Bird being the weakest class (AP 0.7132) despite having 1,056 training instances points to an inherent difficulty: birds appear at highly variable scales, are often small or partially occluded by branches, and their compact silhouette means a small box error produces a large IoU penalty. The size analysis directly confirms this — bird instances skew toward the small and medium buckets where both models struggle most.

Training duration. We trained for 20 epochs rather than the 50 we originally planned, due to Colab session constraints. Early stopping with patience 15 prevented wasted compute, but there is real headroom for improvement: the training loss curves

had not yet fully plateaued at epoch 20. We expect 50 epochs would push mAP@50 above 0.83 for Model A.

Inference speed. Inference ran at approximately 14–16 ms per image on the T4, well within real-time requirements (<33 ms for 30 fps video). This confirms YOLOv8m as a practical deployment choice.

6 Optimisation Strategies

Building on the baseline established by Models A and B, we apply the architectural optimisation proposed by Rasheed and Zarkoosh [1] to produce Model C. The core idea of the paper is that the standard YOLOv8 architecture always runs all three detection heads simultaneously — one targeting small objects ($\leq 32^2$ px), one medium (32^2 – 96^2 px), and one large ($> 96^2$ px) — regardless of whether objects of all sizes are actually present in the dataset. This incurs unnecessary computational cost when a dataset is dominated by a specific size range.

6.1. Object Size Analysis

To identify the dominant object size in our dataset, we implemented the `ObjectSizeClassifier` described in [1], which iterates over all YOLO-format bounding box labels, converts normalised coordinates to pixel area ($\text{area} = w \times W \times h \times H$, where $W = H = 640$), and bins each object into small, medium, or large.

6.2. Modified Architecture: YOLOv8-medium-large

Based on the size analysis, medium and large objects dominate our dataset. We therefore adopt the **YOLOv8-medium-large** variant. This modification removes the small-object detection branch from the neck and head of the original YOLOv8, specifically eliminating the upsample path to the 80×80 feature map and its associated C2f block. The remaining architecture retains the full backbone (layers 0–9) unchanged, and restructures the neck so that the medium head detects at 40×40 resolution and the large head at 20×20 resolution. The result is a two-head detector trained on the same augmentation pipeline as Model A. Training settings for Model C must match Model A exactly (ideally: > 150 epochs) to ensure a fair comparison. While this optimisation demonstrates a meaningful reduction in cost and inference time with minimal accuracy loss, its effectiveness remains data-dependent — A limitation we are still examining and in the next section we will talk about the previous two models.

7 Conclusion

We successfully trained and evaluated a YOLOv8m object detector on a five-class subset of Pascal VOC 2012. Model A, trained with full augmentation (Mosaic, HSV jitter, horizontal flip), achieved mAP@50 of 0.8009 and mAP@50:95 of 0.6087, outperforming the augmentation-free Model B by 4.3 pp and 5.3 pp respectively. Per-class analysis showed cat as the strongest class (AP 0.8687) and bird as the most challenging (AP 0.7132), consistent with the data imbalance and the known difficulty of detecting small, occluded instances. The size-based evaluation confirmed that augmentation matters most for small and medium objects, which is exactly what Mosaic training targets. The controlled experiment confirmed that data augmentation is not optional: it improved every metric and produced more precise bounding boxes as evidenced by the larger gain on mAP@50:95 than on mAP@50. The identified optimisation strategies — extended training, copy-paste augmentation, class-weighted loss, TTA, and SAHI — provide a clear roadmap for pushing the model toward and beyond a 0.83 mAP@50 target.

References

- [1] Areeg Fahad Rasheed and M. Zarkoosh. Optimized YOLOv8 for multi-scale object detection. *Journal of Real-Time Image Processing*, 22(1), 2025.